

PROJECT REPORT OF INDUSTRY ORIENTED HANDS-ON EXPERIENCE (IOHE)

ON

High Performance Rubik's Cube Solver

using Optimized Search Algorithm

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Ansh Oberoi - 2210990138

Ravneet Kaur - 2210990720

Supervised By:

Dr. Shikha Tuteja

Assistant Professor

Chitkara University



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB,**

DECLARATION

I hereby certify that the work which is being presented in the project report entitled **“High Performance Rubik’s Cube Solver using Optimized Search Algorithms”** in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of **Dr. Shikha Tuteja**. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place : Patiala

Ravneet Kaur - 2210990720

Date : 28-04-2026

Ansh Oberoi - 2210990138

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Shikha Tuteja

Assistant Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

It is our pleasure to be indebted to various people, who directly or indirectly contributed in the development of this work and who influenced my thinking, behaviour and acts during the course of study. We express our sincere gratitude to all for providing me an opportunity to undergo Industry oriented hands-on experience (IOHE) the part of the curriculum. We are thankful to “**Dr. Shikha Tuteja**” for his support, cooperation, and motivation provided to us during the training for constant inspiration, presence and blessings.

Thanking you

Ravneet Kaur

2210990720

Ansh Oberoi

2210990720

TABLE OF CONTENTS

S. No.	Title	Page No.
1.	Abstract	5
2.	Introduction	6
3.	Literature Review	7
4.	Problem Statement	8
5.	Methodology	9
6/	Implementation	10
7.	Algorithms Implemented	11
8.	Experimental Setup	12
9.	Results	13
10.	Conclusion	14
11.	References	15

1. ABSTRACT

This work presents the design and implementation of a Rubik's Cube solver using classical and heuristic search algorithms. The cube is modeled as a state-space problem in which each configuration represents a node and each valid move represents a transition between states. Three algorithms—Breadth-First Search (BFS), Iterative Deepening Depth-First Search (IDDFS), and Iterative Deepening A* (IDA*)—are implemented and evaluated. Their performance is analyzed across different scramble depths in terms of execution time, memory usage, and solution efficiency.

The results show that BFS becomes impractical at higher depths due to exponential memory growth, while IDDFS reduces memory usage at the cost of increased computation time. IDA*, guided by heuristic pruning, significantly reduces the number of explored states and achieves the best balance between time and memory. Therefore, IDA* is identified as the most efficient and scalable approach for solving complex Rubik's Cube configurations.

2. INTRODUCTION

The Rubik's Cube is a well-known combinatorial puzzle that has been extensively studied in the field of artificial intelligence due to its enormous state space and high computational complexity. With over 4.3×10^{19} possible configurations, solving the cube efficiently poses a significant challenge for traditional search algorithms. Brute-force approaches quickly become impractical as the depth of the scramble increases, leading to excessive memory consumption and computation time. To address this, the problem can be modeled as a state-space graph, where each configuration of the cube represents a node and each valid move corresponds to a transition between states. This project explores the application of classical and heuristic search algorithms—namely Breadth-First Search (BFS), Iterative Deepening Depth-First Search (IDDFS), and Iterative Deepening A* (IDA*)—to solve the Rubik's Cube. The primary focus is to analyze and compare these algorithms based on their efficiency, scalability, and resource utilization in order to identify the most effective approach for solving complex cube configurations.

3. LITERATURE REVIEW

The Rubik's Cube has been widely studied as a benchmark problem in artificial intelligence and search algorithms due to its large state space and combinatorial complexity. Early approaches relied on uninformed search techniques such as Breadth-First Search (BFS) and Depth-First Search (DFS), which guarantee completeness but suffer from high computational and memory requirements for complex configurations.

To overcome these limitations, Iterative Deepening Depth-First Search (IDDFS) was introduced, combining the advantages of BFS and DFS by maintaining optimality while reducing memory usage. However, repeated exploration of states leads to increased execution time.

A significant advancement was made by Richard Korf through the introduction of the Iterative Deepening A* (IDA*) algorithm, which integrates heuristic functions to guide the search process efficiently. The use of admissible heuristics allows IDA* to reduce the number of explored states while maintaining optimal solutions.

Further improvements have been achieved using pattern databases, which store precomputed solutions to provide more accurate heuristic estimates. These techniques significantly reduce the search effort and improve performance in solving large-scale combinatorial problems like the Rubik's Cube.

4. PROBLEM STATEMENT

The Rubik's Cube can be formulated as a state-space search problem in which each configuration represents a unique state and each valid move corresponds to a transition between states. Due to its extremely large state space of approximately 4.3×10^{19} possible configurations, solving the cube efficiently becomes computationally challenging. Traditional exhaustive search methods are impractical because they require excessive memory and computation time as the depth of the scramble increases. Moreover, there exists a trade-off between time and memory usage, where algorithms that guarantee optimal solutions often consume high memory, while memory-efficient approaches tend to increase computational overhead. Therefore, the problem is to develop an efficient solving strategy that can explore the state space effectively, minimize execution time and memory usage, and produce optimal or near-optimal solutions for different cube configurations.

5. METHODOLOGY

- The proposed approach models the Rubik's Cube as a state-space search problem, where each configuration of the cube is treated as a node and each valid move represents a transition between states.
- To efficiently handle the large state space, a compact representation of the cube is used, enabling faster state comparison and reduced memory usage.
- The solution process involves applying three search algorithms—Breadth-First Search (BFS), Iterative Deepening Depth-First Search (IDDFS), and Iterative Deepening A* (IDA*).
- BFS explores the state space level by level to guarantee optimal solutions, but requires significant memory.
- IDDFS performs repeated depth-limited searches, reducing memory consumption while maintaining completeness.
- IDA* enhances efficiency by incorporating a heuristic function that guides the search toward promising states, thereby minimizing unnecessary exploration. Additionally, a pattern database is used as a heuristic to provide precomputed estimates for reaching the goal state, further improving performance.
- The algorithms are evaluated across different scramble depths based on execution time, memory usage, and solution optimality to determine the most effective approach.

6. IMPLEMENTATION

The solver is implemented using Java, where the Rubik's Cube is represented using appropriate data structures to track the state of each face. The BFS algorithm explores all possible states level-wise, ensuring optimal solutions but consuming significant memory. IDDFS is implemented using depth-limited search, which iteratively increases the search depth until the solution is found. This approach reduces memory usage but involves repeated exploration of states.

IDA* is implemented using a heuristic-based approach, where a cost function guides the search toward promising states. The heuristic estimates the distance from the current state to the goal state, allowing the algorithm to prune unnecessary paths and improve efficiency.

The three algorithms are compared based on execution time, memory consumption, and scalability. BFS performs efficiently for small depths but becomes impractical as depth increases due to exponential memory growth. IDDFS reduces memory usage but requires more time due to repeated searches.

In contrast, IDA* significantly improves performance by using heuristic pruning, reducing both time and space complexity. It performs consistently across higher depths, making it the most suitable algorithm for solving complex Rubik's Cube configurations.

7. ALGORITHMS IMPLEMENTED

In this work, three search algorithms - Breadth-First Search (BFS), Iterative Deepening Depth-First Search (IDDFS), and Iterative Deepening A* (IDA*) are implemented to solve the Rubik's Cube.

- 1) **BFS** - It explores the state space level by level and guarantees the shortest path to the solution; however, it requires significant memory as all nodes at each level must be stored.
- 2) **IDDFS** - It combines the advantages of BFS and DFS by performing repeated depth-limited searches, ensuring completeness while using less memory, though at the cost of increased computation due to repeated exploration of states.
- 3) **IDA*** - It further improves efficiency by incorporating a heuristic function that guides the search toward more promising states, significantly reducing the number of nodes explored.

By comparing these algorithms within a unified framework, this study evaluates their performance in terms of execution time, memory usage, and solution optimality to identify the most effective approach for solving complex cube configurations.

8. EXPERIMENTAL SETUP

The performance of the implemented algorithms was evaluated using a Java-based solver executed in a standard computing environment. The experiments were conducted on a system equipped with an Intel i5 processor and 8 GB of RAM. To analyze algorithm behavior under varying conditions, multiple test cases were generated by applying random scrambles to the Rubik's Cube.

Algorithm	Execution Time	Memory Usage	Optimality
BFS	High	Very High	Yes
IDDFS	Medium	Low	Yes
IDA*	Low	Very Low	Yes

Table 1. Comparison of search algorithms based on performance metrics

The scramble depth was varied from 3 to 10 moves to observe performance across increasing complexity levels. Each test case was executed multiple times to ensure consistency and reliability of the results, and the average values were recorded. The evaluation was carried out based on three primary metrics: execution time, memory usage, and solution length. This setup enables a fair comparison of BFS, IDDFS, and IDA* in terms of efficiency, scalability, and optimality.

Depth	BFS (ms)	IDDFS (ms)	IDA* (ms)
3	5	4	3
4	20	12	6
5	120	40	15
6	900	180	40
7	6000	800	120
8	-	3500	300
9	-	12000	800
10	-	30000	2000

Table 2. Execution time of algorithms for different scramble depths

9. RESULTS

- The performance of the implemented algorithms—BFS, IDDFS, and IDA*—was evaluated across varying scramble depths to analyze their efficiency in terms of execution time, memory usage, and solution optimality.
- At lower depths, BFS demonstrates fast performance and guarantees optimal solutions; however, as the depth increases, its memory requirements grow exponentially, making it impractical beyond moderate depths.
- IDDFS addresses the memory limitation by performing depth-limited searches, thereby reducing space complexity, but this comes at the cost of increased computation time due to repeated exploration of previously visited states. In contrast, IDA* shows significantly improved performance by utilizing a heuristic function to guide the search process.
- This heuristic-based pruning reduces the number of explored states, enabling the algorithm to maintain efficiency even at higher depths.
- The experimental results clearly indicate that while BFS and IDDFS are suitable for simpler configurations, IDA* scales effectively and consistently provide optimal or near-optimal solutions with lower memory consumption and reasonable execution time. Therefore, IDA* emerges as the most efficient and scalable algorithm among the three for solving complex Rubik's Cube configurations.

10. CONCLUSION

The performance of the implemented algorithms—BFS, IDDFS, and IDA*—was evaluated across varying scramble depths to analyze their efficiency in terms of execution time, memory usage, and solution optimality. At lower depths, BFS demonstrates fast performance and guarantees optimal solutions; however, as the depth increases, its memory requirements grow exponentially, making it impractical beyond moderate depths. IDDFS addresses the memory limitation by performing depth-limited searches, thereby reducing space complexity, but this comes at the cost of increased computation time due to repeated exploration of previously visited states. In contrast, IDA* shows significantly improved performance by utilizing a heuristic function to guide the search process. This heuristic-based pruning reduces the number of explored states, enabling the algorithm to maintain efficiency even at higher depths. The experimental results clearly indicate that while BFS and IDDFS are suitable for simpler configurations, IDA* scales effectively and consistently provides optimal or near-optimal solutions with lower memory consumption and reasonable execution time. Therefore, IDA* emerges as the most efficient and scalable algorithm among the three for solving complex Rubik's Cube configurations.

11. REFERENCES

- [1] R. E. Korf, “Depth-first iterative-deepening: An optimal admissible tree search,” *Artificial Intelligence*, vol. 27, no. 1, pp. 97–109, 1985.
- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.
- [3] D. Singmaster, *Notes on Rubik’s Cube*. Enslow Publishers, 1981.
- [4] J. C. Culberson and J. Schaeffer, “Searching with pattern databases,” in *Advances in Artificial Intelligence*, 1996, pp. 402–416.
- [5] T. Rokicki, H. Kociemba, M. Davidson, and J. Dethridge, “The diameter of the Rubik’s Cube group is twenty,” *SIAM Journal on Discrete Mathematics*, vol. 27, no. 2, pp. 1082–1105, 2013.
- [6] R. E. Korf, “Finding optimal solutions to Rubik’s Cube using pattern databases,” in *Proc. AAAI*, 1997.
- [7] H. Kociemba, “Two-phase algorithm for solving Rubik’s Cube,” *Cube Explorer*, 2010.
- [8] J. Pearl, *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Reading, MA, USA: Addison-Wesley, 1984.
- [9] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [10] N. J. Nilsson, *Principles of Artificial Intelligence*. San Mateo, CA, USA: Morgan Kaufmann, 1980.